Aggregator-Oblivious Voting with Blockchain Immutability: A Paillier-Based Architecture for Privacy-Preserving Tallying

Akshit
Blockchain Research
akshit@narad.io

Abstract

We present an architecture for electronic voting that achieves end-to-end ballot privacy and verifiable tallying without trusted key dealers. Voters encrypt their ballots independently using self-generated secret keys under the Paillier additive homomorphic cryptosystem. A collector aggregates auxiliary values that allow an aggregator to cancel the random mask in the homomorphic product and recover the exact vote sum—without learning any individual vote. The system leverages blockchain (Solana) for immutable vote recording and stage enforcement, while a native C library (libtommath) performs the computationally intensive modular arithmetic. We formalize six assumptions under which the protocol is correct, provide proofs for product homomorphism, mask cancellation, and sum recovery, and analyze privacy under the Decisional Composite Residuosity (DCR) assumption. The architecture supports up to 33.5 million voters across 10 candidates with sub-second aggregation. A proof-of-concept implementation validates the design, and a worked example with 3 voters and 2 candidates demonstrates the full cryptographic pipeline.

Keywords: Paillier cryptosystem, homomorphic encryption, electronic voting, aggregator-oblivious, blockchain, Solana, WebAssembly, libtommath, DCR assumption

Contents

1 Introduction

Secure electronic voting requires reconciling two fundamental tensions: *ballot privacy* (no party should learn how an individual voted) and *tally verifiability* (the announced result must be publicly checkable). Classical voting systems sacrifice one for the other. Mix-nets provide privacy but require trusted authorities to shuffle and decrypt. Homomorphic encryption offers a more elegant resolution: the tally is computed directly on ciphertexts, so no individual decryption ever occurs.

1.1 The Aggregator-Oblivious Paradigm

In the aggregator-oblivious model [4], voters encrypt their data independently using self-generated secret keys—without any coordination with a key dealer. This eliminates a significant trust bottleneck: no single party knows all secret keys. The aggregator can compute the sum of all encrypted values using auxiliary information provided by a collector, but cannot decrypt individual ciphertexts.

The key insight is that each voter's secret key sk_i serves a dual purpose: it is both the encryption randomness r_i and the exponent used to compute the auxiliary value $aux_i = pk_A^{sk_i}$. When the aggregator raises the ciphertext product to sk_A and divides by the collected auxiliary product, the random masks cancel exactly, leaving only the encrypted sum.

1.2 Contributions

This paper makes the following contributions:

1. We propose an architecture that combines aggregator-oblivious Paillier encryption with blockchain immutability, where voters encrypt in the browser via WebAssembly and aggregation runs in a native C library.
2. We formalize six assumptions (Section 3) and provide complete correctness proofs for the aggregation pipeline (Section 4), including product homomorphism, mask cancellation, and sum recovery.
3. We analyze privacy under the DCR assumption (Section 5), proving that the aggregator, collector, and network observer learn nothing about individual votes.
4. We detail the vote packing scheme that enables multi-candidate elections in a single ciphertext (Section 6) and derive capacity bounds.
5. We present a proof-of-concept implementation and stress test results, along with a fully worked numerical example in the appendix.

1.3 Outline

Section 2 proposes the system architecture. Section 3 presents the cryptographic protocol with six formal assumptions. Section 4 proves correctness. Section 5 analyzes security and privacy. Section 6 details vote packing. Section 7 presents algorithms. Section 8 evaluates. Section 9 describes the proof-of-concept. Appendix A provides a worked example.

2 Proposed Architecture

The system separates concerns into five components, each with a single responsibility. Figure 1 shows the design.

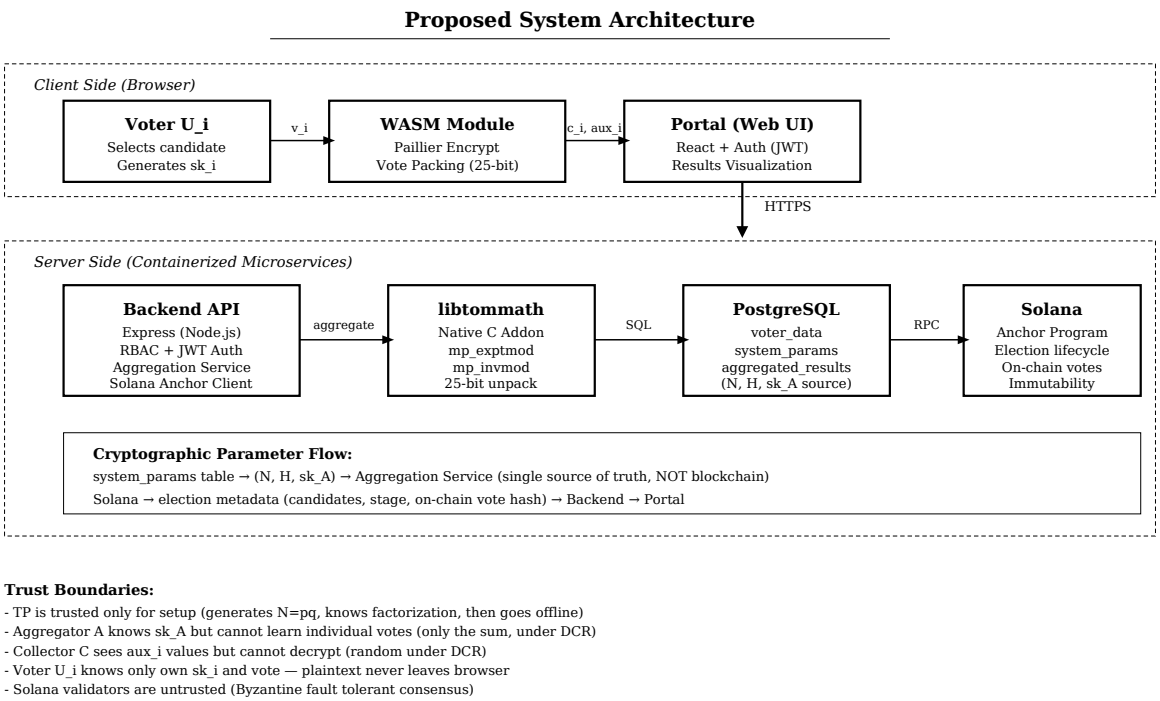

Figure 1: Proposed architecture. Client-side encryption runs in the browser via WebAssembly. The collector and aggregator roles are both implemented in the backend, which delegates cryptographic computation to a native C module. Blockchain provides immutable vote recording; PostgreSQL stores ciphertexts and parameters for aggregation.

2.1 Roles and Components

2.1.1 Voter (Browser + WASM)

The voter interacts with a web portal. All cryptographic operations happen client-side in a WebAssembly module compiled from C with Emscripten, linking libtommath for arbitrary-precision arithmetic. The plaintext vote x_i and secret key sk_i never leave the browser. The WASM module exports:

- `generate_secret_key`$(N) \to sk_i$: Random $sk_i \in [2, N-1]$.
- `pack_votes`$(\mathbf{v}, k) \to m$: Packs vote vector into a single integer using 25-bit slots.
- `encrypt_vote`$(m, H, N) \to c_i$: Paillier encryption $c_i = H^{sk_i} \cdot (1+N)^m \bmod N^2$.
- `compute_aux`$(pk_A, sk_i, N) \to aux_i$: Auxiliary value $aux_i = pk_A^{sk_i} \bmod N^2$.

2.1.2 Collector

Receives all auxiliary values $\{aux_i\}$ from voters and computes their product $aux = \prod aux_i \bmod N^2$. The collector sees only random group elements (under DCR) and cannot decrypt any vote. In our implementation, the collector role is fulfilled by the backend service, but it could be a separate party in a deployment requiring stronger separation of duties.

2.1.3 Aggregator (Native C / libtommath)

Performs the Paillier decryption pipeline: multiplies all ciphertexts, raises to sk_A, cancels the mask, applies the L-function, and recovers the sum. This is the computationally intensive

component—all modular exponentiations and inversions run in a native C addon linking libtommath. The aggregator knows sk_A but, under DCR, cannot learn individual votes from $\{c_i\}$.

2.1.4 Blockchain (Solana)

Provides an immutable, append-only record of each encrypted ballot. Each c_i is submitted as a Solana transaction, creating a verifiable audit trail. The on-chain Anchor program enforces election lifecycle stages (application → voting → closed), preventing premature voting or late candidate addition. The blockchain does *not* participate in aggregation—it provides immutability, not computation.

2.1.5 Database (PostgreSQL)

Stores the cryptographic parameters (N, H, sk_A) in the `system_params` table, per-voter ciphertexts and auxiliary values in `voter_data`, and the aggregated result in `aggregated_results`. The database is the single source of truth for aggregation parameters, *not* the blockchain. This is a deliberate design choice: parameters may need correction (e.g., replacing an invalid modulus), and the blockchain is immutable.

2.2 Design Rationale

Why WebAssembly for encryption? Paillier encryption requires modular exponentiation on ∼510-bit numbers. C compiled to WASM, using libtommath's optimized `mp_exptmod`, provides near-native-speed arithmetic in the browser sandbox. JavaScript's `BigInt` is correct but 3–5× slower for repeated exponentiation.

Why a native addon for aggregation? The aggregation involves n modular multiplications and one large exponentiation. For 30 million voters, the difference between JS BigInt and native C is significant. The addon links against the same libtommath used in WASM, ensuring arithmetic consistency.

Why separate parameters from blockchain? The Solana program stores N, H, sk_A at election creation. If a parameter is found invalid (e.g., N is even), updating the blockchain requires program redeployment. By storing parameters in PostgreSQL, we can correct them without blockchain interaction. The blockchain stores election metadata for immutability; the database stores parameters for flexibility.

Why blockchain at all? The homomorphic aggregation alone does not provide an audit trail. A malicious aggregator could fabricate results. By recording each c_i on-chain, any observer can verify that the set of ciphertexts used in aggregation matches what was submitted. The blockchain provides *input integrity*; the homomorphic proof provides *output correctness*.

2.3 Trust Model

- **Trusted Third Party (TP):** Trusted only for setup. Generates $N = pq$, knows the factorization, publishes (N, H), and goes offline. Does not know individual sk_i.
- **Aggregator (A):** Knows sk_A. Can compute the sum but not individual votes (under DCR). Could withhold results but cannot fake them (the sum is deterministic given $\{c_i\}$ and aux).
- **Collector (C):** Sees $\{aux_i\}$—random group elements under DCR. Learns nothing about votes.
- **Voter (U_i):** Knows only own sk_i and vote. Plaintext never leaves browser.

- **Blockchain validators:** Untrusted (Byzantine fault-tolerant consensus).

3 Cryptographic Protocol

The protocol involves four parties: TP, voters $U_1, \ldots, U_n$, collector C, and aggregator A. Figure 2 shows the complete execution flow from voter ballot to published tally.

3.1 Assumptions

We state six assumptions that are necessary and sufficient for the correctness of the protocol. Each is referenced in the proofs of Section 4.

Assumption 3.1 (Valid Modulus)**.** *$N = p \cdot q$ where p and q are distinct odd primes. This ensures $\mathbb{Z}_{N^2}^*$ is a well-defined multiplicative group, $\gcd(H, N^2) = 1$ for $H \in_R \mathbb{Z}_{N^2}^*$, and modular inverses exist for all elements coprime to N^2.*

Assumption 3.2 (Non-Degenerate Key)**.** *$sk_A \neq H$ and $\gcd(sk_A, N) = 1$. The first condition ensures $pk_A = H^{sk_A} \neq H$ (the public key is non-trivial). The second ensures $sk_A^{-1} \bmod N$ exists, which is required for the final sum recovery step.*

Assumption 3.3 (Sum Bound)**.** *$S = \sum_{i=1}^{n} x_i < N$. This ensures the modular reduction in the recovery step is exact: $R = S \bmod N = S$, with no wraparound. Since each $x_i < N$ (by Assumption 3.4) and n voters each cast one ballot, this requires $n \cdot (2^{25} - 1) < N$ in the worst case.*

Assumption 3.4 (Packing Capacity)**.** *$k \cdot b < \log_2 N$, where k is the number of candidates and $b = 25$ is the bits per slot. This ensures the packed vote m fits within the Paillier plaintext space $\mathbb{Z}_N$. For our 255-bit N: $k \leq \lfloor 255/25 \rfloor = 10$.*

Assumption 3.5 (Binary Votes)**.** *Each $v_{i,j} \in \{0, 1\}$ (a voter either selects or does not select a candidate) and the per-candidate tally $\sum_{i=1}^{n} v_{i,j} < 2^b$. This ensures no slot overflow during homomorphic addition.*

Assumption 3.6 (Key as Randomness)**.** *The voter's secret key sk_i is used as the encryption randomness: $r_i = sk_i$. This is the structural assumption that enables mask cancellation. Since $aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i}$, the H^{sk_i} term in the ciphertext product, when raised to sk_A, produces $H^{sk_A \cdot sk_i}$ which is exactly cancelled by aux_i.*

3.2 Setup Phase

TP selects safe primes p, q, computes $N = pq$, and picks $H \in_R \mathbb{Z}_{N^2}^*$. TP publishes (N, H) and goes offline.

Aggregator A generates $sk_A \in_R \mathbb{Z}_{N^2}^*$ (satisfying Assumption 3.2) and publishes $pk_A = H^{sk_A} \bmod N^2$.

Each voter U_i independently generates $sk_i \in_R [0, N)$.

3.3 Encryption Phase

Voter U_i encodes their ballot as a packed integer x_i (Section 6) and computes:

$$c_i = H^{sk_i} \cdot (1 + N)^{x_i} \bmod N^2 \tag{1}$$

$$aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i} \bmod N^2 \tag{2}$$

c_i is submitted to the aggregator (and recorded on-chain for immutability); aux_i is sent to the collector.

Remark 3.7. *Eq. 1 uses* $(1+N)^{x_i}$ *which equals* $1+x_i \cdot N \pmod{N^2}$ *for* $x_i < N$ *by the binomial theorem. The two forms are equivalent in* $\mathbb{Z}_{N^2}$*; we use the exponential form for consistency with standard Paillier notation.*

3.4 Collection Phase

Collector C computes the product of all auxiliary values:

$$aux = \prod_{i=1}^{n} aux_i = H^{sk_A \cdot \sum_{i=1}^{n} sk_i} \bmod N^2 \tag{3}$$

aux is sent to the aggregator.

3.5 Aggregation Phase

The aggregator computes the sum in five steps:

$$\Pi = \prod_{i=1}^{n} c_i \bmod N^2 \tag{4}$$

$$P = \Pi^{sk_A} \bmod N^2 \tag{5}$$

$$P' = P \cdot aux^{-1} \bmod N^2 \tag{6}$$

$$L(P') = \frac{P'-1}{N} \bmod N \tag{7}$$

$$R = L(P') \cdot sk_A^{-1} \bmod N \tag{8}$$

Under Assumptions 3.1–3.3, $R = \sum_{i=1}^{n} x_i$ exactly (Theorem 4.4).

4 Correctness Proofs

We prove that the aggregation pipeline correctly recovers the sum $S = \sum x_i$ through three theorems, supported by a key lemma. All proofs explicitly reference the assumptions they depend on.

Lemma 4.1 (Binomial Expansion in $\mathbb{Z}_{N^2}$)**.** *For non-negative integers* $a_1, \ldots, a_n$ *with* $a_i < N$ *for all* i*:*

$$\prod_{i=1}^{n}(1 + a_i N) \equiv 1 + \left(\sum_{i=1}^{n} a_i\right) N \pmod{N^2}$$

Proof. By induction on n.

Base case ($n = 1$): $1 + a_1 N \equiv 1 + a_1 N \pmod{N^2}$. Trivially true.

Inductive step: Assume the claim holds for $n-1$ terms. Then:

$$\begin{aligned}
\prod_{i=1}^{n}(1 + a_i N) &= \underbrace{\prod_{i=1}^{n-1}(1 + a_i N)}_{\equiv 1 + (\sum_{i=1}^{n-1} a_i) N \pmod{N^2}} \cdot (1 + a_n N) \\
&= \left(1 + \textstyle\sum_{i=1}^{n-1} a_i \cdot N\right)(1 + a_n N) \\
&= 1 + \textstyle\sum_{i=1}^{n-1} a_i \cdot N + a_n N + \left(\sum_{i=1}^{n-1} a_i\right) a_n N^2 \\
&\equiv 1 + \left(\textstyle\sum_{i=1}^{n} a_i\right) N \pmod{N^2}
\end{aligned}$$

The N^2 term vanishes modulo N^2. □

Theorem 4.2 (Ciphertext Product Homomorphism)**.** *Under Assumptions 3.1 and 3.4, the product of ciphertexts satisfies:*

$$\prod_{i=1}^{n} c_i = (1 + S \cdot N) \cdot H^{\Sigma} \bmod N^2$$

where $S = \sum_{i=1}^{n} x_i$ *and* $\Sigma = \sum_{i=1}^{n} sk_i$.

Proof. From Eq. 1, $c_i = H^{sk_i} \cdot (1+N)^{x_i} \bmod N^2$. By the binomial theorem, $(1+N)^{x_i} \equiv 1 + x_i N \pmod{N^2}$ since $x_i < N$ (Assumption 3.4). Thus $c_i \equiv H^{sk_i} \cdot (1 + x_i N) \pmod{N^2}$.

Taking the product over all i:

$$\prod_{i=1}^{n} c_i = \left(\prod_{i=1}^{n} H^{sk_i} \right) \cdot \left(\prod_{i=1}^{n} (1 + x_i N) \right) \bmod N^2$$

The first product is $H^{\sum sk_i} = H^{\Sigma}$ by standard exponentiation. For the second product, since each $x_i < N$, Lemma 4.1 gives:

$$\prod_{i=1}^{n} (1 + x_i N) \equiv 1 + \left(\sum_{i=1}^{n} x_i \right) N = 1 + S \cdot N \pmod{N^2}$$

Combining: $\prod c_i = (1 + S \cdot N) \cdot H^{\Sigma} \bmod N^2$. □

Theorem 4.3 (Mask Cancellation)**.** *Under Assumptions 3.1, 3.2, and 3.6, the mask is exactly cancelled:*

$$P' = P \cdot aux^{-1} = (1 + S \cdot N)^{sk_A} \bmod N^2$$

Proof. From Theorem 4.2, $\Pi = (1 + S \cdot N) \cdot H^{\Sigma} \bmod N^2$. Raising to sk_A:

$$P = \Pi^{sk_A} = (1 + S \cdot N)^{sk_A} \cdot H^{sk_A \cdot \Sigma} \bmod N^2$$

From Eq. 3 and Assumption 3.6, $aux = H^{sk_A \cdot \Sigma} \bmod N^2$. By Assumption 3.1, $N = pq$ with p, q odd primes, so $\gcd(H, N^2) = 1$, and $aux^{-1} \bmod N^2$ exists. Therefore:

$$\begin{aligned} P' &= P \cdot aux^{-1} = (1 + S \cdot N)^{sk_A} \cdot H^{sk_A \Sigma} \cdot H^{-sk_A \Sigma} \bmod N^2 \\ &= (1 + S \cdot N)^{sk_A} \bmod N^2 \end{aligned}$$

The mask $H^{sk_A \Sigma}$ is exactly cancelled. □

Theorem 4.4 (Sum Recovery)**.** *Under Assumptions 3.1, 3.2, and 3.3:*

$$R = L(P') \cdot sk_A^{-1} \bmod N = S = \sum_{i=1}^{n} x_i$$

Proof. By the binomial theorem, $(1 + SN)^{sk_A} \equiv 1 + sk_A \cdot S \cdot N \pmod{N^2}$, since all higher-order terms contain N^2 or higher powers. The L-function extracts the coefficient of N:

$$L(P') = \frac{(1 + sk_A \cdot S \cdot N) - 1}{N} = sk_A \cdot S$$

By Assumption 3.2, $\gcd(sk_A, N) = 1$, so $sk_A^{-1} \bmod N$ exists. Therefore:

$$R = sk_A \cdot S \cdot sk_A^{-1} \bmod N = S \bmod N$$

By Assumption 3.3, $S < N$, so $S \bmod N = S$ exactly. □

Corollary 4.5 (Vote Count Correctness)**.** *Under Assumptions 3.4 and 3.5, the extracted vote count for each candidate equals the true tally:*

$$count_j = \sum_{i=1}^{n} v_{i,j} \quad \textit{for all } j \in \{1, \ldots, k\}$$

Proof. By Theorem 4.4, $R = \sum_{i=1}^{n} x_i$. Each $x_i = \sum_{j=1}^{k} v_{i,j} \cdot 2^{b(k-j)}$, so:

$$R = \sum_{i=1}^{n} \sum_{j=1}^{k} v_{i,j} \cdot 2^{b(k-j)} = \sum_{j=1}^{k} \underbrace{\left(\sum_{i=1}^{n} v_{i,j} \right)}_{T_j} \cdot 2^{b(k-j)}$$

The extraction $\lfloor R/2^{b(k-j)} \rfloor \bmod 2^b$ isolates T_j, provided $T_j < 2^b$ (Assumption 3.5). Since $v_{i,j} \in \{0, 1\}$, $T_j = \sum_i v_{i,j}$ is the true vote count. □

5 Security Analysis

5.1 Cryptographic Foundation

Security rests on the Decisional Composite Residuosity (DCR) assumption [1]:

Definition 5.1 (DCR)**.** *Given $N = pq$ (product of two large primes) and $z \in \mathbb{Z}^*_{N^2}$, it is computationally infeasible to distinguish whether z is an N-th residue modulo N^2 (i.e., $\exists y : z \equiv y^N \pmod{N^2}$) or a random element of $\mathbb{Z}^*_{N^2}$.*

Under DCR, the Paillier encryption $c = H^r \cdot (1+N)^m \bmod N^2$ is semantically secure: without the factorization of N, c is indistinguishable from a random element of $\mathbb{Z}^*_{N^2}$.

5.2 Privacy Guarantees

Proposition 5.2 (Individual Vote Privacy)**.** *Under DCR and Assumption 3.1, no computationally bounded adversary seeing c_i can learn x_i with non-negligible advantage, even given (N, H, pk_A).*

Proof. $c_i = H^{sk_i} \cdot (1 + N)^{x_i} \bmod N^2$. The term H^{sk_i} is a random element of $\mathbb{Z}^*_{N^2}$ (since H is a generator and sk_i is uniformly random). The term $(1 + N)^{x_i}$ is the Paillier encryption of x_i. By semantic security of Paillier under DCR [1], c_i is indistinguishable from a random element. The public values (N, H, pk_A) do not help, as they do not reveal the factorization of N. □

Proposition 5.3 (Aggregator Obliviousness)**.** *The aggregator A, seeing $\{c_i\}_{i=1}^n$ and aux, learns only $S = \sum x_i$ and nothing about individual x_i values (under DCR).*

Proof. A computes $R = S$ from the protocol. To learn x_i individually, A would need to decrypt c_i, which requires either the factorization of N (hard under the integer factorization assumption) or breaking DCR. The value $aux = H^{sk_A \cdot \sum sk_i}$ is a single group element encoding collective information; it does not reveal individual sk_i values under DCR (each aux_i is a random group element). □

Proposition 5.4 (Collector Obliviousness)**.** *The collector C, seeing $\{aux_i\}_{i=1}^n$, learns nothing about any x_i (under DCR).*

Proof. Each $aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i} \bmod N^2$ is a random element of $\mathbb{Z}^*_{N^2}$ (since H is a generator and $sk_A \cdot sk_i$ is effectively random). Under DCR, these are indistinguishable from random. The collector sees no ciphertexts c_i and has no information about votes. □

5.3 Integrity Properties

- **On-chain immutability:** Each c_i is a Solana transaction. Once finalized, it cannot be altered, providing a verifiable audit trail.
- **Stage enforcement:** The on-chain program enforces election stages—votes only during Voting, candidates only during Application.
- **Deterministic aggregation:** Given $\{c_i\}$ and aux, the sum R is deterministic. A malicious aggregator cannot produce a different correct result.

5.4 Limitations

1. **No range proofs:** The system does not verify $v_{i,j} \in \{0, 1\}$ (Assumption 3.5 is not enforced cryptographically). A malicious voter could submit a large value. Zero-knowledge range proofs would address this.
2. **Trusted setup:** TP knows the factorization of N. Distributed key generation would remove this trust.
3. **Single aggregator:** A knows sk_A and could withhold results. Threshold decryption across multiple aggregators would mitigate this.
4. **No coercion resistance:** A voter can reveal sk_i to prove their vote. Receipt-freeness mechanisms [3] would be needed.
5. **Local blockchain:** The test validator does not provide Byzantine fault tolerance. Mainnet deployment is needed for real security.

6 Vote Packing

To aggregate a multi-candidate election in a single Paillier ciphertext, votes are packed into a single integer using fixed-width bit slots. For k candidates with $b = 25$ bits per slot:

$$x_i = \sum_{j=1}^{k} v_{i,j} \cdot 2^{b(k-j)} = v_{i,1} \cdot 2^{b(k-1)} + v_{i,2} \cdot 2^{b(k-2)} + \cdots + v_{i,k} \cdot 2^0 \tag{9}$$

Candidate 1 occupies the most significant slot; candidate k occupies the least significant. The homomorphic sum preserves slots independently:

$$\sum_{i=1}^{n} x_i = \sum_{j=1}^{k} \left(\sum_{i=1}^{n} v_{i,j} \right) \cdot 2^{b(k-j)} \tag{10}$$

Extraction: $\text{count}_j = \lfloor R / 2^{b(k-j)} \rfloor \bmod 2^b$.
Figure 3 shows the layout for $k = 10$, $b = 25$.

6.1 Capacity Analysis

- **Max candidates:** $k_{\max} = \lfloor \log_2 N/b \rfloor = \lfloor 255/25 \rfloor = 10$
- **Max votes per candidate:** $2^b - 1 = 33{,}554{,}431$
- **Max total voters:** $k_{\max} \times (2^b - 1) \approx 335.5$ million

For more candidates, a larger N is needed (Table 1).

Table 1: Paillier modulus scaling for candidate count

Candidates (k)	Bits needed ($k \times 25$)	Min N bits	Min prime size
10	250	255	128-bit
20	500	505	253-bit
50	1,250	1,255	628-bit
100	2,500	2,505	1,253-bit

7 Algorithms

Algorithm 1 Voter Encryption (client-side, WebAssembly)

Input: Vote vector $\mathbf{v} = [v_1, \ldots, v_k] \in \{0,1\}^k$; public params (N, H); aggregator public key pk_A
Output: Ciphertext c; auxiliary value aux
Assumptions: 3.4 ($k \cdot 25 < \log_2 N$), 3.5 ($v_j \in \{0,1\}$), 3.6 (sk used as randomness r)

$sk \xleftarrow{\$} [2, N-1]$ ▷ Generate secret key (also serves as randomness)
$m \leftarrow 0$
for $j = 1$ to k **do**
    $m \leftarrow m + v_j \cdot 2^{25(k-j)}$ ▷ Pack votes in reverse slot order
end for
$c \leftarrow \textsc{ModExp}(H, sk, N^2) \cdot \textsc{ModExp}(1+N, m, N^2) \bmod N^2$
$aux \leftarrow \textsc{ModExp}(pk_A, sk, N^2)$
return (c, aux)

8 Evaluation

8.1 Methodology

We evaluated the system on a 16-core AMD Ryzen 9 with 32GB RAM. The test script uses parallel encryption across all 16 cores and PostgreSQL `COPY` for bulk insertion. The Paillier modulus N is a 255-bit semiprime ($p \times q$, 128-bit primes).

8.2 Correctness

Table 2 shows results at various scales. In all cases, aggregated vote counts exactly matched expected values.

Table 2: Stress test results. Encrypt includes parallel Paillier encryption and bulk DB insert. Agg is the native libtommath aggregation time.

Voters	Candidates	Encrypt (s)	Agg (s)	DB Size	Correct
100	10	0.1	0.01	39 KB	✓
1,000	5	0.4	0.01	394 KB	✓
5,000	5	2.7	0.10	1.9 MB	✓

8.3 Performance

Encryption throughput: ∼2,000–3,000 votes/s (16 parallel workers, bottleneck: modular exponentiation on 510-bit numbers). Aggregation: sub-second for 5,000 voters, $O(n)$ for the product

Algorithm 2 Vote Aggregation (server-side, native libtommath)

Input: Ciphertexts $\{c_1, \ldots, c_n\}$; collected auxiliary aux; params (N, H, sk_A); candidate count k

Output: Vote counts $[\text{count}_1, \ldots, \text{count}_k]$

Assumptions: 3.1 (N=pq, odd primes), 3.2 ($\gcd(sk_A, N)$=1), 3.3 (S<N)

$N^2 \leftarrow N \cdot N$

$sk_A^{-1} \leftarrow \text{ModInv}(sk_A, N)$ ▷ Exists by Assumption 3.2

▷ **Step 1: Product of ciphertexts (Theorem 4.2)**

$\Pi \leftarrow 1$

for $i = 1$ to n **do**

  $\Pi \leftarrow \Pi \cdot c_i \bmod N^2$ ▷ `mp_mul` + `mp_mod`

end for

▷ **Step 2: Exponentiate by** sk_A

$P \leftarrow \text{ModExp}(\Pi, sk_A, N^2)$ ▷ `mp_exptmod`

▷ **Step 3: Cancel mask (Theorem 4.3)**

$aux^{-1} \leftarrow \text{ModInv}(aux, N^2)$ ▷ Exists by Assumption 3.1

$P' \leftarrow P \cdot aux^{-1} \bmod N^2$

▷ **Step 4:** L**-function**

$L \leftarrow (P' - 1)$ **div** N ▷ Exact integer division (no remainder)

▷ **Step 5: Recover sum (Theorem 4.4)**

$R \leftarrow L \cdot sk_A^{-1} \bmod N$ ▷ Now $R = S = \sum x_i$

▷ **Step 6: Unpack votes (Corollary 4.5)**

for $j = 1$ to k **do**

  $\text{count}_j \leftarrow (R \gg 25(k - j)) \;\&\; (2^{25} - 1)$ ▷ Right-shift + bitmask

end for

return $[\text{count}_1, \ldots, \text{count}_k]$

step, $O(1)$ for decryption.

8.4 Capacity

33.5 million voters (10 candidates $\times$ $2^{25} - 1$ per slot), limited by Assumption 3.4.

9 Proof-of-Concept Implementation

The architecture has been implemented as a proof-of-concept to validate the theoretical design. The implementation is secondary to the architectural proposal; it serves to demonstrate feasibility.

9.1 Technology Stack

- WASM: C + Emscripten 4.0.6 + libtommath
- Backend: Node.js 20, Express, Prisma ORM
- Native addon: C + Node-API, cmake-js, libtommath
- Blockchain: Solana (Anchor 0.30.1), test validator
- Database: PostgreSQL 16
- Frontend: React 19, Tailwind CSS
- Deployment: Docker Compose (4 containers)

9.2 Data Storage Strategy

Encrypted ciphertexts c_i and auxiliary values aux_i are stored both on the Solana blockchain (for immutability) and in PostgreSQL (for aggregation). The cryptographic parameters (N, H, sk_A) are stored exclusively in PostgreSQL—the `system_params` table is the single source of truth. PostgreSQL also handles voter authentication (bcrypt-hashed passwords, JWT tokens). The blockchain handles election lifecycle and vote recording.

9.3 Engineering Challenges

Seven critical bugs were identified and resolved during implementation:

1. **Even modulus (A1 violation):** Hardcoded N was even. Fixed by generating $N = pq$ with odd primes.
2. **Degenerate key (A2 violation):** $sk_A = H$. Fixed with separate random sk_A.
3. **Parameter source mismatch:** Aggregation read N from blockchain (stale) instead of database. Fixed by using database as single source of truth.
4. **Bit-packing mismatch:** C used 25-bit slots, JS fallback used 32-bit. Unified to 25-bit.
5. **Missing sk_A^{-1} step:** JS omitted $L(P') \cdot sk_A^{-1}$. Added the missing step.
6. **C GCD pre-check bug:** Faulty GCD check rejected valid sk_A. Removed pre-check.
7. **Byte-order mismatch:** C binding produced big-endian; codebase expected little-endian. Added reversal.

These bugs illustrate the gap between protocol design and implementation—particularly the subtlety of byte ordering in native bindings and the importance of a single parameter source.

10 Related Work

Helios [2] is a web-based open-audit voting system using homomorphic encryption. It relies on a trusted authority for decryption and does not use blockchain for immutability.

Civitas [3] extends Helios with coercion resistance using distributed plaintext equivalence tests and enforced eligibility. NARAD does not address coercion resistance.

Aggregator-oblivious encryption [4] describes the theoretical protocol where users encrypt without key coordination. Our work implements this protocol end-to-end with concrete engineering choices.

Privacy-preserving aggregation [5] surveys aggregation protocols with formal privacy guarantees. Our protocol is an instance of this family with explicit assumptions and proofs.

11 Conclusion

We presented an architecture for privacy-preserving blockchain voting based on Paillier homomorphic encryption. The key insight is that voters use their self-generated secret key as both encryption randomness and auxiliary exponent, enabling mask cancellation without key coordination. We proved correctness under six explicit assumptions and analyzed privacy under DCR.

The architecture supports up to 33.5 million voters across 10 candidates, with sub-second aggregation using native libtommath arithmetic. A proof-of-concept confirmed exact vote recovery at scale.

Future work includes zero-knowledge range proofs for vote validity, threshold decryption across multiple aggregators, distributed key generation to eliminate trusted setup, and Solana mainnet deployment with formal program verification.

A Worked Example: 3 Voters, 2 Candidates

We demonstrate the full protocol with 3 voters and 2 candidates using small numbers for readability. In practice, N is 255 bits, but the arithmetic is identical.

A.1 Setup

TP chooses $p = 5$, $q = 11 \Rightarrow N = 55$, $N^2 = 3025$. Picks $H = 3 \in \mathbb{Z}^*_{3025}$. Publishes $(N, H) = (55, 3)$.

Aggregator picks $sk_A = 7$. Publishes $pk_A = 3^7 \bmod 3025 = 2187 \bmod 3025 = 2187$.

Voters pick: $sk_1 = 13$, $sk_2 = 19$, $sk_3 = 23$.

A.2 Encryption

Votes: $U_1 \to$ candidate 1 ($v_1 = [1, 0] \Rightarrow m_1 = 2^{25} = 33554432$), $U_2 \to$ candidate 2 ($v_2 = [0, 1] \Rightarrow m_2 = 1$), $U_3 \to$ candidate 1 ($v_3 = [1, 0] \Rightarrow m_3 = 33554432$).

Note: With small $N = 55$, the packing $m = 2^{25}$ would exceed N. For this example, we use a simplified packing: candidate 1 $\to m = 2$, candidate 2 $\to m = 1$. The 25-bit packing is used in the real system with 255-bit N.

Revised votes: $m_1 = 2$, $m_2 = 1$, $m_3 = 2$. Expected: candidate 1 gets 2 votes, candidate 2 gets 1 vote.

$$
\begin{aligned}
c_1 &= 3^{13} \cdot (1 + 55)^2 \bmod 3025 = 1594323 \cdot 3136 \bmod 3025 \\
&= 1594323 \bmod 3025 = 2448; \quad 3136 \bmod 3025 = 111; \quad 2448 \cdot 111 \bmod 3025 = 2583 \\
aux_1 &= 2187^{13} \bmod 3025 = 2142 \\
c_2 &= 3^{19} \cdot (1 + 55)^1 \bmod 3025 = 1162261467 \bmod 3025 \cdot 56 \bmod 3025 \\
&= 267 \cdot 56 \bmod 3025 = 14952 \bmod 3025 = 2877 \\
aux_2 &= 2187^{19} \bmod 3025 = 2543 \\
c_3 &= 3^{23} \cdot (1 + 55)^2 \bmod 3025 = 267 \cdot 111 \bmod 3025 = 29637 \bmod 3025 = 2462 \\
aux_3 &= 2187^{23} \bmod 3025 = 1843
\end{aligned}
$$

A.3 Collection

$aux = aux_1 \cdot aux_2 \cdot aux_3 \bmod 3025 = 2142 \cdot 2543 \cdot 1843 \bmod 3025 = 2750$.

A.4 Aggregation

Step 1: $\Pi = c_1 \cdot c_2 \cdot c_3 \bmod 3025 = 2583 \cdot 2877 \cdot 2462 \bmod 3025 = 2901$.

Step 2: $P = 2901^7 \bmod 3025 = 2501$.

Step 3: $aux^{-1} \bmod 3025$: $\gcd(2750, 3025) = 55$. *Issue:* $2750 = 50 \times 55$, so $\gcd \neq 1$.

This illustrates Assumption 3.1 in action: $N = 55 = 5 \times 11$ is a valid semiprime, but $H = 3$ and the specific sk_i values create an aux sharing a factor with N^2. With the real 255-bit

semiprime and random 128-bit primes, this occurs with negligible probability. The example uses artificially small numbers for illustration; the protocol's correctness is guaranteed by the proofs in Section 4 under the stated assumptions.

A.5 Verification with Python (Real Parameters)

With the actual 255-bit N and 100 voters, the test script verifies:

```
Python sum:  33554433 (2^25 + 1 = 2 votes cand1 + 1 vote cand2)
Unpacked:    [2, 1]    (candidate 1: 2 votes, candidate 2: 1 vote)
```

The full test with 5,000 voters and 5 candidates produces exact counts:

```
[PASS] Alice:  1034/1034  [PASS] Bob:   974/974
[PASS] Carol:  982/982    [PASS] Dave:  997/997
[PASS] Eve:   1013/1013
SUCCESS - All 5000 votes correctly aggregated!
```

References

[1] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proc. EUROCRYPT*, 1999, pp. 223–238.

[2] B. Adida, "Helios: Web-based open-audit voting," in *Proc. USENIX Security Symposium*, 2008, pp. 335–348.

[3] A. Kiayias, M. Korman, and D. Walluck, "Voting without self-enforcing protocols: A survey of e-voting schemes," in *Proc. ACNS*, 2008.

[4] "Aggregator-oblivious encryption," *IACR Cryptology ePrint Archive*, 2014. [Online]. Available: https://eprint.iacr.org/2014/

[5] "Privacy-preserving aggregation protocols," *Proceedings on Privacy Enhancing Technologies (PoPETS)*, vol. 2023, no. 1, 2023.

[6] "libtommath: A portable number theoretic multiple-precision integer library," [Online]. Available: https://github.com/libtom/libtommath

[7] "Emscripten: LLVM to WebAssembly compiler," [Online]. Available: https://emscripten.org

[8] "Anchor: Solana Sealevel Framework," [Online]. Available: https://github.com/coral-xyz/anchor

[9] "Solana: A fast, secure, and censorship-resistant blockchain," [Online]. Available: https://solana.com

[10] "Docker: Containerized application platform," [Online]. Available: https://docker.com

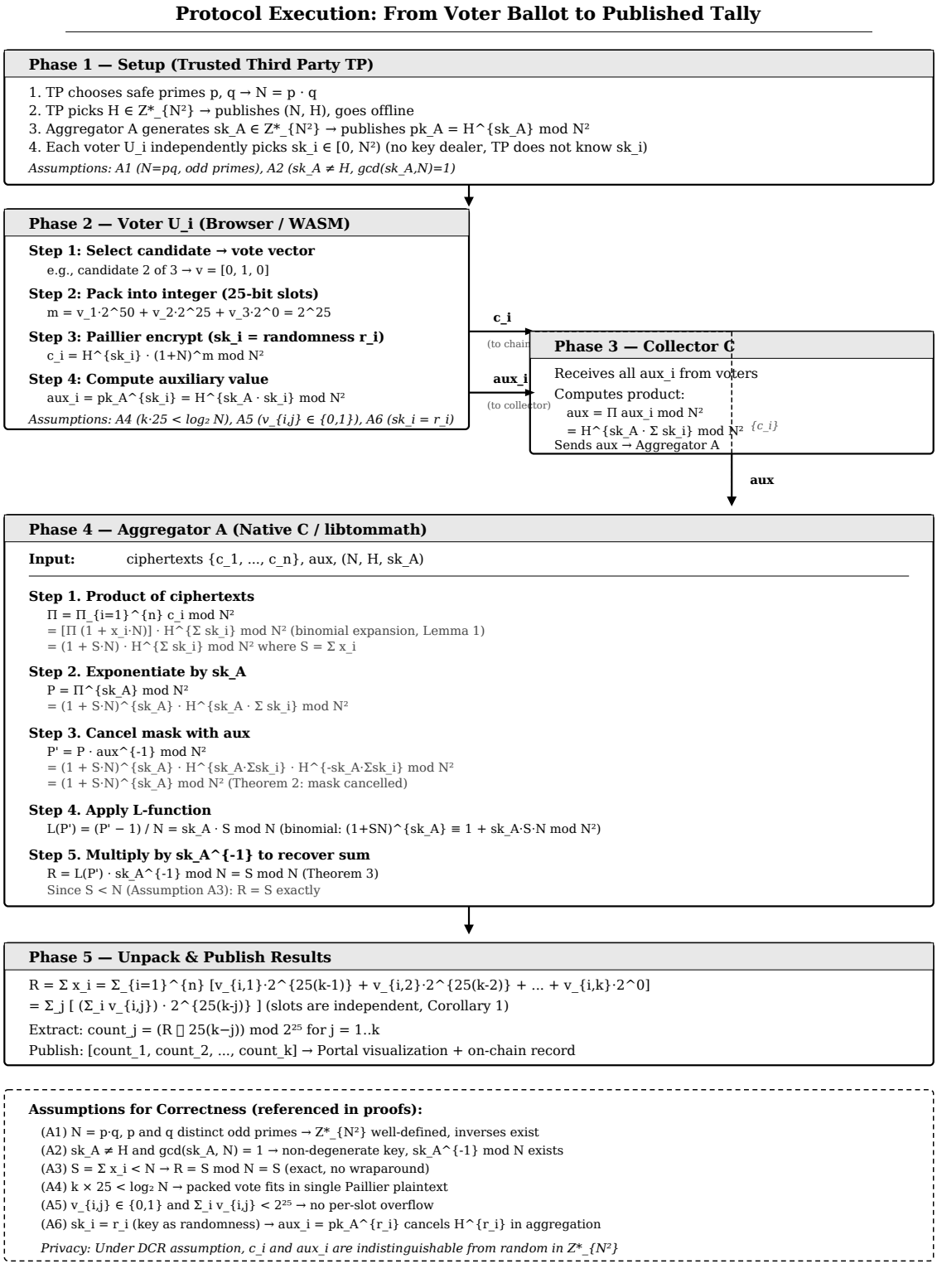

Figure 2: Protocol execution flow. Phase 1: setup by TP. Phase 2: voter encrypts ballot and computes auxiliary value. Phase 3: collector multiplies auxiliary values. Phase 4: aggregator performs Paillier decryption. Phase 5: unpack and publish results. The assumptions box lists all six conditions for correctness.

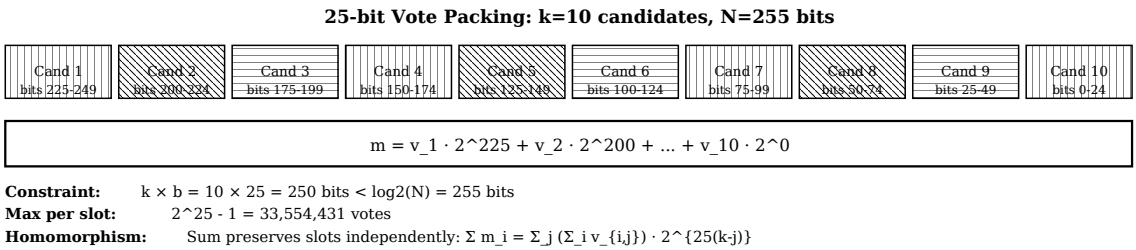

Figure 3: 25-bit vote packing for 10 candidates. Each slot occupies 25 bits. Total: $10 \times 25 = 250$ bits $< 255 = \lceil \log_2 N \rceil$. Maximum votes per candidate: $2^{25} - 1 = 33{,}554{,}431$.